TRIBHUVAN UNIVERSITY

Institute of Engineering, Purwanchal Campus

Department of Electronics and Computer Engineering

Dharan, Sunsari

ARTIFICIAL INTELLIGENCE [ENCT 305]

LAB REPORT: TO IMPLEMENT NEURAL REPRESENTATION OF AND AND X-OR GATE USING PERCEPTRON.

Submitted By

Abhishek Niraula

Roll no.: PUR080BEI004

Faculty: BEI

Submitted To

Er. Sujan Karki

Instructor

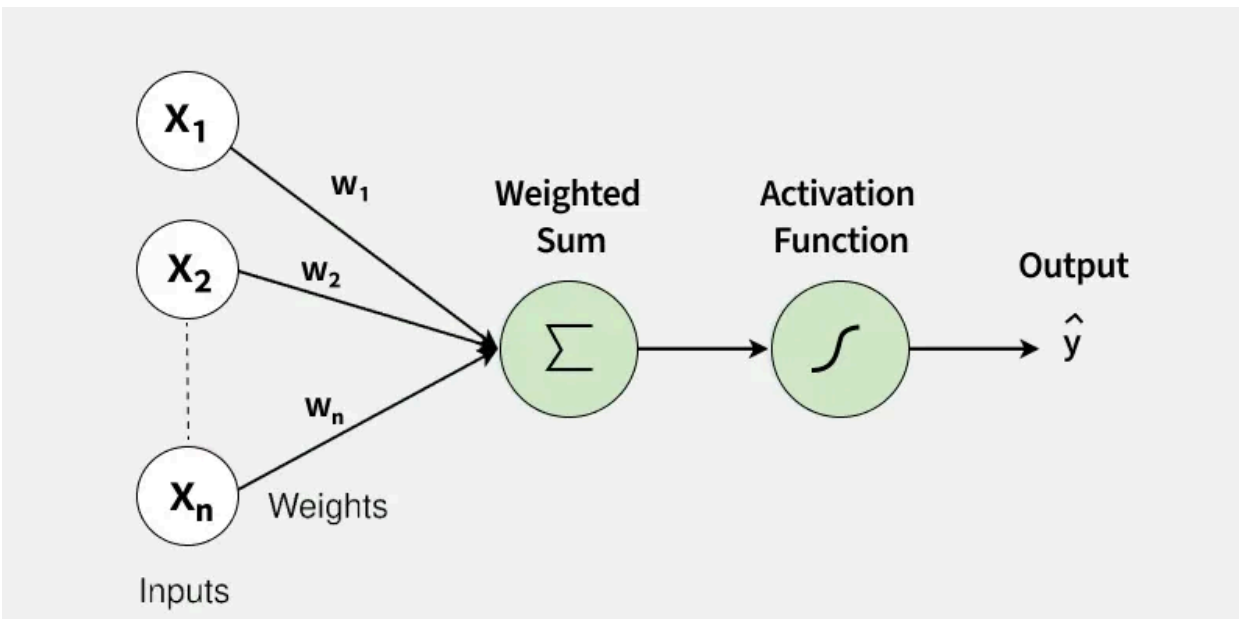
Purwanchal Campus, Dharan

INTRODUCTION

A **Perceptron** is the simplest form of a neural network that makes decisions by combining inputs with weights and applying an activation function. It was invented in **1957** by **Frank Rosenblatt.** It is mainly used for binary classification problems. It forms the basic building block of many deep learning models.

- Takes multiple inputs and assigns weights
- Computes a weighted sum and applies a threshold
- Outputs either 0 or 1 (binary outcome)
- Forms the foundation of larger neural networks

Core Components



A) Inputs (x_1, x_2, x_3, ..., x_n)

These are the features or measurable attributes of a data point that the perceptron uses to make a decision. Each input provides a signal that contributes to the final output.

- **Example:** For an OR gate, the inputs are binary: $(x_1, x_2) \in \{0, 1\}^2$
- Inputs themselves have no inherent influence unless multiplied by weights.

B) Weights (w_1, w_2, w_3, ..., w_n)

Weights determine how strongly each input contributes to the prediction. A larger weight means the corresponding input has a higher impact.

- Weights are learned during training, adjusting based on errors.
- They act like importance scores for each feature.

C) Bias (b)

The bias is a constant value added to the weighted sum to shift the decision boundary.

It allows the perceptron to classify correctly even when all input features are zero.

Bias ensures the model is not forced to pass the decision boundary through the origin.

Difference Between Weights and Bias

Difference Between Weights and Bias

- Weights control how much each input influences the output.
- Bias controls when the perceptron activates, independent of any input.
- Mathematically, Weights tilt the line and Bias shifts the line up/down or left/right.

D) Net Input (Weighted Sum)

This is the combined effect of all inputs and their weights:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- Represents the activation strength before passing through the activation function.
- If z is high or low enough, it determines the final class.

E) Activation Function (Step Function)

The activation function converts the numerical input into a binary output:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- Represents the activation strength before passing through the activation function.
- If z is high or low enough, it determines the final class.

OBJECTIVES

A) To understand the concept of Perceptron and its working as a basic unit of neural networks.

B) To implement the perceptron logic in **Python** to classify linearly separable data (**AND Gate**).

C) To know the limitations of **Single Layer Perceptron** and its inability to solve XOR Gate problem

D) To use **MultiLayer Perceptron** to classify complex training data (**XOR Gate**).

E) To understand backpropagation and updation of weights and biases in neural networks training.

ALGORITHM (Working of Perceptron)

1. **Initialize Parameters**

   Set weights w_1, w_2, ..., w_n and bias b to small values (often zero).

2. **Compute Weighted Sum**

   For each input vector $x = (x_1, x_2, ..., x_n)$, calculate:

3. **Apply Activation Function**

   - Use the step function:

   $$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

4. **Update Weights and Bias (Learning Rule)**

   - For each training example, compute error:

     $error = y_{true} - \bar{y}$

   - Update parameters:

     $w_i = w_i + \eta * error * x_i$

     $b = b + \eta * error$

     Where, η is the learning rate.

5. **Repeat for Epochs**

   Iterate over the dataset multiple times until the perceptron correctly classifies all inputs or reaches the maximum number of epochs.

AND GATE Classification using Single Layer Perceptron

The AND gate is a fundamental logic gate in digital electronics. It outputs **1** only when **both inputs are 1**, otherwise the output is **0**. Its truth table is:

Input 1	Input 2	Output
0	0	0
0	1	0
1	0	0
1	1	1

This makes the **AND gate** a simple binary classification problem, where the perceptron must learn to separate the single positive case (1,1 → 1) from the three negative cases. The perceptron adjusts its weights and bias during training until it can correctly classify all input combinations.

CODE - AND GATE

main.py

```
import numpy as np
import pandas as pd
import streamlit as st
import matplotlib.pyplot as plt
from model import Perceptron

st.title("AND Logic Implementation Using Perceptron")
st.markdown("---")

x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])

df = pd.DataFrame(x, columns = ['Input 1', 'Input 2'])
df['Output'] = y
st.header("DATASET")
st.table(df)

st.sidebar.header("Training HyperParameters")
learning_rate = st.sidebar.slider("Learning Rate", 0.01, 1.0, 0.1)
```

```
epoch = st.sidebar.slider("Epoch", 1, 5000, 1000)

w1 = st.sidebar.number_input("Weight 1", value = 0.0, step = 0.1)
w2 = st.sidebar.number_input("Weight 2", value = 0.0, step = 0.1)
b = st.sidebar.number_input("Bias", value = 0.0, step = 0.1) #Bias

if "model" not in st.session_state:
    st.session_state.model = None

if st.button("Train Model"):
    model = Perceptron(lr = learning_rate, epoch = epoch, weight =
np.array([w1, w2]), bias = b)
    model.fit(x, y)
    st.session_state.model = model
    st.success("Model Trained Successfully")

if st.session_state.model is not None:
    model = st.session_state.model
    st.header("Final Parameters")
    st.write(f"Weights: {model.weight}")
    st.write(f"Bias: {model.bias}")
    st.header("Predictions")
    predictions = model.predict(x)
    result = pd.DataFrame(x, columns=["Input 1", "Input 2"])
    result['Predicted'] = predictions

    st.table(result)
    st.header("Test Your Predictions")
    col1, col2 = st.columns(2)

    with col1:
        input1 = st.selectbox("Select First Input", [0, 1], key = "input1")

    with col2:
        input2 = st.selectbox("Select Second Input", [0, 1], key = "input2")

    pred = model.predict([[input1, input2]])
    st.info(f"The predicted value is {pred[0]}")
```

model.py

```
import numpy as np

class Perceptron:

    def __init__(self, lr = 0.01, epoch = 100, weight = None, bias = None):
```

```
        self.lr = lr
        self.epoch = epoch
        self.bias = bias
        self.weight = weight

    def step_function(self, z):
        if z >= 0:
            return 1
        else:
            return 0

    def fit(self, x, y):
        n_sample, n_features = x.shape
        self.weight = np.zeros(n_features)
        self.bias = 0

        for e in range(self.epoch):
            for i in range(n_sample):
                z = np.dot(x[i], self.weight) + self.bias
                y_pred = self.step_function(z)
                error = y[i] - y_pred
                self.weight += self.lr * error * x[i]
                self.bias += self.lr * error

    def predict(self, x):
        prediction = []
        for sample in x:
            z = np.dot(sample, self.weight) + self.bias
            y_pred = self.step_function(z)
            prediction.append(y_pred)
        return prediction
```

OUTPUT – AND GATE

Training HyperParameters

Learning Rate
0.10

Epoch
1000

Weight 1
0.10

Weight 2
0.10

Bias
0.10

AND Logic Implementation Using Perceptron

DATASET

	Input 1	Input 2	Output
0	0	0	0
1	0	1	0
2	1	0	0
3	1	1	1

Train Model

Final Parameters

Weights: [0.2 0.1]

Bias: -0.20000000000000004

Predictions

	Input 1	Input 2	Predicted
0	0	0	0
1	0	1	0
2	1	0	0
3	1	1	1

Test Your Predictions

Select First Input

1

Select Second Input

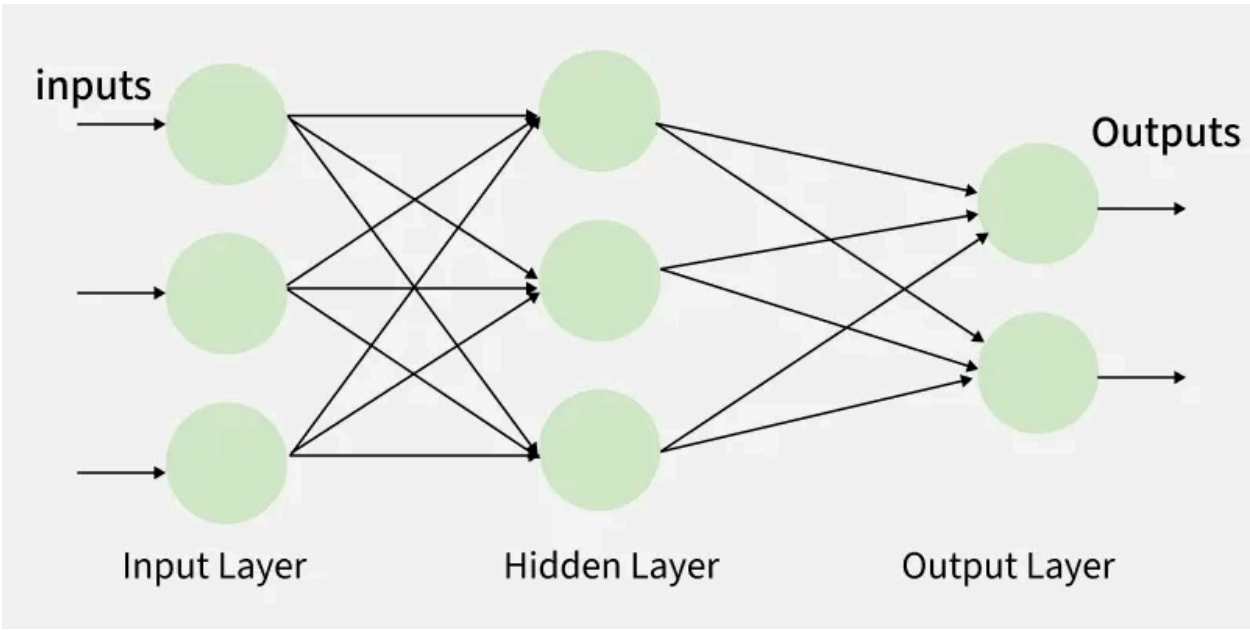
1

The predicted value is 1

MULTILAYER PERCEPTRON

Multi-Layer Perceptron (MLP) consists of fully connected dense layers that transform input data from one dimension to another. It is called multi-layer because it contains an input layer, one or more hidden layers and an output layer. The purpose of an MLP is to model complex relationships between inputs and outputs.

Core Components



- **Input Layer:** Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers:** MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer:** The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.

Working of a MultiLayer Perceptron

A) Forward Propagation

In forward propagation the data flows from the input layer to the output layer, passing through any hidden layers. Each neuron in the hidden layers processes the input as follows:

1) **Weighted Sum**

   The neuron computes the weighted sum of the inputs:

$$z = \sum_i w_i x_i + b$$

Where,

- x_i is the input feature.
- w_i is the corresponding weight.
- b is the bias term.

2) **Activation Function**
   The weighted sum z is passed through an activation function to introduce non-linearity. Common activation functions include: Sigmoid, ReLU, Tanh, etc.

B) Loss Function

Once the network generates an output the next step is to calculate the loss using a loss function. In supervised learning this compares the predicted output to the actual label.

C) Backpropagation

The goal of training an MLP is to minimize the loss function by adjusting the network's weights and biases. This is achieved through **backpropagation**:

- **Gradient Calculation:** The gradients of the loss function with respect to each weight and bias are calculated using the chain rule of calculus.
- **Error Propagation:** The error is propagated back through the network, layer by layer.
- **Gradient Descent:** The network updates the weights and biases by moving in the opposite direction of the gradient to reduce the loss:

  $$w = w - \eta . \frac{\delta L}{\delta w}$$

  Where,

  - w is the weight.
  - η is the learning rate.
  - $\frac{\delta L}{\delta w}$ is the gradient of the loss function with respect to the weight.

D) Optimization

MLPs rely on optimization algorithms to iteratively refine the weights and biases

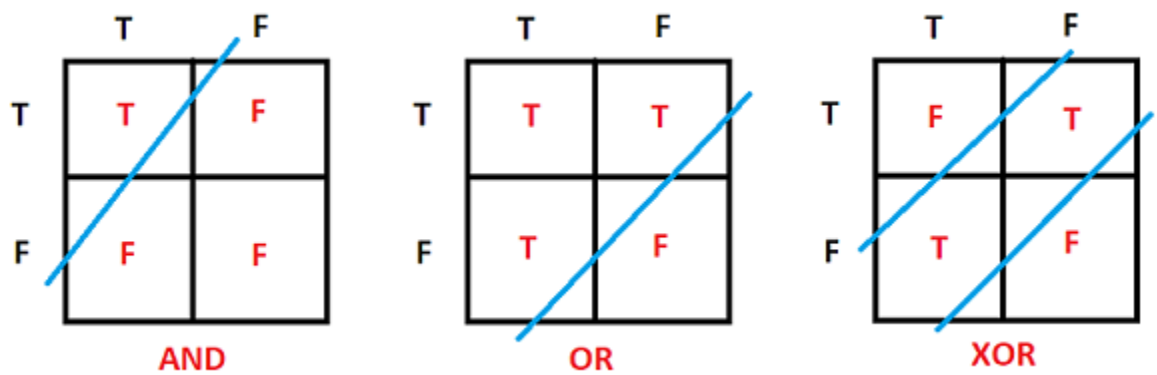
during training.

XOR GATE Classification using MultiLayer Perceptron

The **XOR gate** outputs 1 only when the two inputs differ (i.e., one is 0 and the other is 1), and outputs 0 when both inputs are the same. Unlike the AND gate, XOR is not linearly separable, meaning no single straight line can separate the input-output classes in a two-dimensional space.

To solve this, a **multilayer perceptron** introduces hidden layers with nonlinear activation functions. These hidden neurons transform the input space into a higher-dimensional representation where the XOR classes become separable. The MLP learns intermediate features that capture the nonlinearity of the XOR function, allowing the network to correctly classify all four input combinations.

Why Single Layer Perceptron cannot solve the XOR Problem?



A **single-layer perceptron** cannot solve **the XOR problem** because XOR is not linearly separable. In the case of the AND or OR gates, it is possible to draw a straight line in the input space (with axes representing the two inputs) that separates the output classes (0 and 1). However, for XOR, the positive outputs (when inputs differ) and negative outputs (when inputs are the same) are arranged diagonally in the input space. No single straight line can separate these two classes correctly.

Since a perceptron only computes a weighted sum of inputs and applies a threshold, it can only represent linear decision boundaries. XOR requires a **non-linear boundary**, which cannot be achieved with just one layer. This is why a **multilayer perceptron (MLP)** with hidden neurons is needed: the hidden layer transforms the input space into a

higher-dimensional representation, allowing the network to construct a non-linear boundary that correctly classifies XOR inputs.

This limitation of the single-layer perceptron was historically significant—it showed that while perceptrons are powerful for linearly separable problems, more complex architectures (like MLPs) are essential for solving non-linear problems such as **XOR**.

CODE - XOR GATE

main.py

```
import numpy as np
import pandas as pd
import streamlit as st
from model import XORPerceptron

st.title("XOR Logic Implementation Using Multi-Layer Perceptron")
st.markdown("---")

x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])

df = pd.DataFrame(x, columns = ['Input 1', 'Input 2'])
df['Output'] = y
st.header("DATASET")
st.table(df)

st.sidebar.header("Training HyperParameters")
learning_rate = st.sidebar.slider("Learning Rate", 0.01, 1.0, 0.1)
epoch = st.sidebar.slider("Epoch", 1, 5000, 1000)

if "model" not in st.session_state:
    st.session_state.model = None

if st.button("Train Model"):
    model = XORPerceptron(lr = learning_rate, epoch = epoch)
    model.fit(x, y)
    st.session_state.model = model
    st.success("Model Trained Successfully")

if st.session_state.model is not None:
    model = st.session_state.model
    st.header("Final Parameters")
    st.write(f"Perceptron OR - Weights: {model.perceptron_or.weight}, Bias:
```

```
{model.perceptron_or.bias}")
    st.write(f"Perceptron NAND - Weights: {model.perceptron_nand.weight}, Bias:
{model.perceptron_nand.bias}")
    st.write(f"Perceptron AND - Weights: {model.perceptron_and.weight}, Bias:
{model.perceptron_and.bias}")
    st.header("Predictions")
    predictions = model.predict(x)
    result = pd.DataFrame(x, columns=["Input 1", "Input 2"])
    result['Predicted'] = predictions

    st.table(result)
    st.header("Test Your Predictions")
    col1, col2 = st.columns(2)

    with col1:
        input1 = st.selectbox("Select First Input", [0, 1], key = "input1")

    with col2:
        input2 = st.selectbox("Select Second Input", [0, 1], key = "input2")

    pred = model.predict([[input1, input2]])
    st.info(f"The predicted value is {pred[0]}")
```

model.py

```
import numpy as np

class Perceptron:

    def __init__(self, lr = 0.01, epoch = 100, weight = None, bias = None):
        self.lr = lr
        self.epoch = epoch
        self.bias = bias
        self.weight = weight

    def step_function(self, z):
        if z >= 0:
            return 1
        else:
            return 0

    def fit(self, x, y):
        n_sample, n_features = x.shape
        self.weight = np.zeros(n_features)
        self.bias = 0
```

```
        for e in range(self.epoch):
            for i in range(n_sample):
                z = np.dot(x[i], self.weight) + self.bias
                y_pred = self.step_function(z)
                error = y[i] - y_pred
                self.weight += self.lr * error * x[i]
                self.bias += self.lr * error

    def predict(self, x):
        prediction = []
        for sample in x:
            z = np.dot(sample, self.weight) + self.bias
            y_pred = self.step_function(z)
            prediction.append(y_pred)
        return prediction


# Logic: XOR = (x1 OR x2) AND (NOT(x1 AND x2))
# Simplified Logic: XOR = (x1 OR x2) AND (x1 NAND x2)
class XORPerceptron:
    def __init__(self, lr = 0.01, epoch = 100):
        self.lr = lr
        self.epoch = epoch
        self.perceptron_nand = Perceptron(lr = lr, epoch = epoch)
        self.perceptron_or = Perceptron(lr = lr, epoch = epoch)\

        # Perceptron_and is special : It is for the hidden layer. It performs
and to the output of the hidden layer i.e perceptron_nand and perceptron_or
        self.perceptron_and = Perceptron(lr = lr, epoch = epoch)

    def fit(self, x, y):
        # Resultand array of or and nand operation
        y_or = np.array([0, 1, 1, 1])
        y_nand = np.array([1, 1, 1, 0])
        self.perceptron_nand.fit(x, y_or)
        self.perceptron_or.fit(x, y_nand)
        # Hidden Layer
        hidden_outut = []
        for i in range(len(x)):

            # Z1 and Z2 are the perceptron output logic before passing it to
the step function i.e z = w1x1 + w2x2 + b
            z1 = np.dot(x[i], self.perceptron_nand.weight) +
self.perceptron_nand.bias
            z2 = np.dot(x[i], self.perceptron_or.weight) +
```

```
self.perceptron_or.bias

            out1 = self.perceptron_nand.step_function(z1)
            out2 = self.perceptron_or.step_function(z2)

            # Input to the third perceptron layer is the outputs of NAND and OR
gates.
            hidden_outut.append([out1, out2])

        hidden_outut = np.array(hidden_outut)
        self.perceptron_and.fit(hidden_outut, y)

    # Same prediction logic as single Perceptron
    def predict(self, x):
        prediction = []
        for sample in x:
            z1 = np.dot(sample, self.perceptron_nand.weight) +
self.perceptron_nand.bias
            out1 = self.perceptron_nand.step_function(z1)

            z2 = np.dot(sample, self.perceptron_or.weight) +
self.perceptron_or.bias
            out2 = self.perceptron_or.step_function(z2)

            hidden = np.array([out1, out2])
            z3 = np.dot(hidden, self.perceptron_and.weight) +
self.perceptron_and.bias
            y_pred = self.perceptron_and.step_function(z3)

            prediction.append(y_pred)

        return prediction
```

OUTPUT – XOR GATE

Training HyperParameters

Learning Rate
0.10

Epoch
1000

XOR Logic Implementation Using Multi-Layer Perceptron

DATASET

	Input 1	Input 2	Output
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	0

Train Model

Model Trained Successfully

Final Parameters

Perceptron OR - Weights: [-0.2 -0.1], Bias: 0.2

Perceptron NAND - Weights: [0.1 0.1], Bias: -0.1

Perceptron AND - Weights: [0.1 0.2], Bias: -0.20000000000000004

Predictions

	Input 1	Input 2	Predicted
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	0

Test Your Predictions

Select First Input

1

Select Second Input

0

The predicted value is 1

CONCLUSION

In this lab, we implemented the perceptron model to solve logical gate problems, specifically the **AND** and **XOR** gates. For the AND gate, the perceptron successfully learned the correct classification since the problem is linearly separable. We observed how weights and bias were updated during training and how the perceptron converged to produce accurate outputs.

When extended to the XOR gate, however, the single-layer perceptron failed because XOR is not linearly separable. This highlighted the limitation of simple perceptrons and the need for **multilayer perceptrons (MLPs)** with hidden layers to handle non-linear problems. Through this experiment, we gained practical experience in coding perceptron models, visualizing datasets, and understanding the role of linear separability in classification tasks.